

Linguaggio Macchina

Linguaggio macchina

Un programma in linguaggio macchina è una serie di istruzioni codificate.

Esempio: istruzione in linguaggio macchina a 16 bit è la seguente 1010001100011001.

Per capire cosa significa questa istruzione dobbiamo capire il meccanismo con cui questa istruzione è stata creata, esempio:

- Il linguaggio dice che le istruzioni sono divise in campi da 4 bit
- Il linguaggio dice che i primi 4 bit a sinistra dicono quale istruzione la CPU dovrà eseguire
- I restanti tre campi dicono quali operandi sono coinvolti.

Linguaggio macchina

Esempio di istruzione in linguaggio macchina

1010 | 0011 | 0001 | 1001

Istruzione da eseguire R3 R1 R9

Supponendo di aver costruito adeguatamente la nostra macchina, un'istruzione di questo tipo potrebbe avere il seguente significato:

1010 | 0011 | 0001 | 1001

ADD R3 R1 R9

Somma il contenuto del registro R1 al contenuto del registro R9 ed inserisci il risultato in R3.

Linguaggio macchina

Dato che scrivere codice binario potrebbe essere piuttosto critico, il linguaggio macchina viene spesso associato a simboli mnemonici (etichette, ad esempio la ADD).

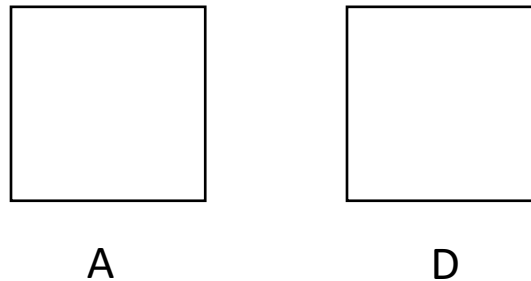
Chi progetta la CPU deve progettare anche il linguaggio ed è a sua discrezione associare un simbolo mnemonico ad un numero binario (esempio: a 1010 associo ADD).

Con questo meccanismo una persona può scrivere 1010001100011001 oppure ADD R3,R1,R9.

Il linguaggio Assembly

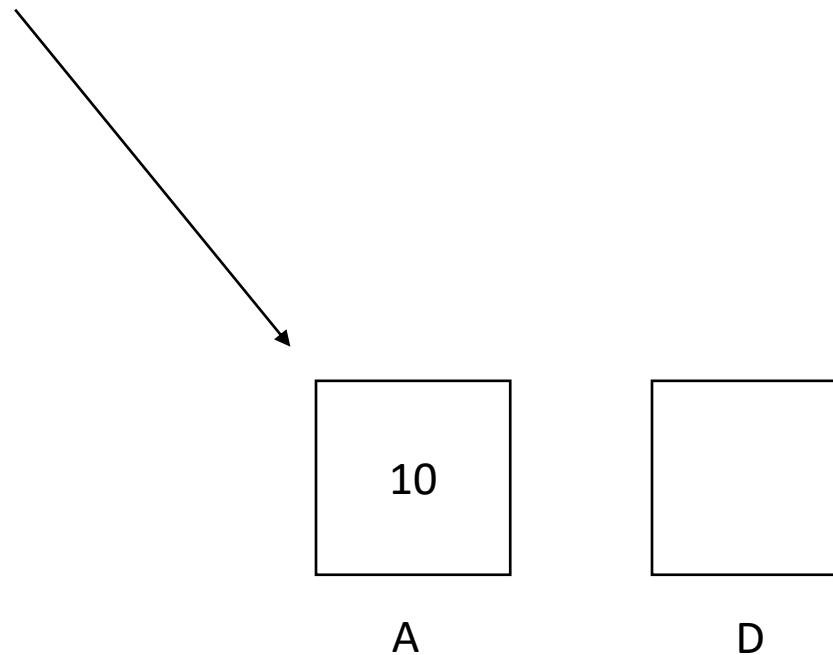
Prima di iniziare a scrivere in linguaggio macchina dobbiamo aver ben chiaro in mente come essa funziona.

La nostra macchina è formata da due registri A (utilizzato per tenere in memoria gli indirizzi) e D (utilizzato per tenere in memoria i dati).



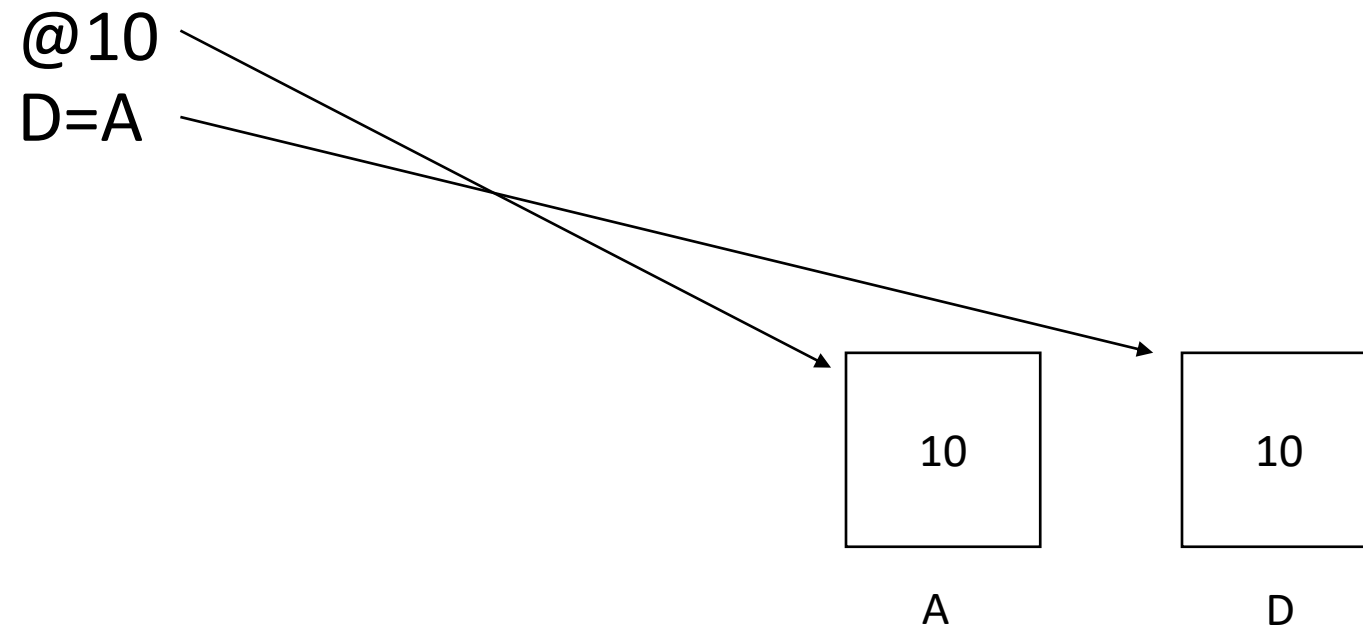
Il linguaggio Assembly

E' possibile inserire una costante solamente nel registro A con l'istruzione: @10



Il linguaggio Assembly

E' possibile assegnare al registro D una costante solamente inserendola prima nel registro A e poi copiandola nel registro D:



Il linguaggio Assembly

Con la nostra CPU le operazioni consentite sono:

<i>comp</i>	<i>mnemonic</i>			<i>comp</i>	<i>mnemonic</i>
	0				
	1				
	-1				
	D				
	A			M	
	!D				
	!A			!M	
	-D				
	-A			-M	
	D+1				
	A+1			M+1	
	D-1				
	A-1			M-1	
	D+A			D+M	
	D-A			D-M	
	A-D			M-D	
	D&A			D&M	
	D A			D M	

Il linguaggio Assembly

Gestione della memoria:

Andremo ad utilizzare due memorie distinte: ROM e RAM.

La ROM conterrà solamente le istruzioni e non può essere scritta in alcun modo (read only) mentre la RAM conterrà i dati pertanto essa potrà essere scritta tutte le volte che lo si ritiene necessario.

Indirizzo	Valore
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	10
8	0
9	0
...	...

MEMORIA ROM

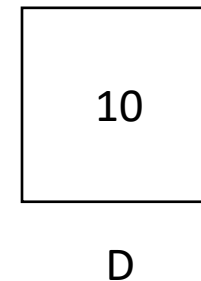
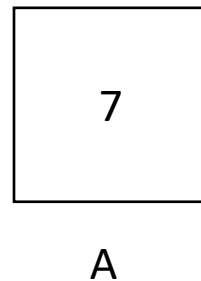
Indirizzo	Valore
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	10
8	0
9	0
...	...

MEMORIA RAM

Il linguaggio Assembly

E' possibile copiare il valore del registro D in memoria. L'indirizzo di memoria deve essere scritto nel registro A e si fa riferimento alla memoria con la lettera M:

@7
M=D



M=D

Indirizzo	Valore
0	0
1	0
2	0
3	0
4	0
5	0
6	0
7	10
8	0
9	0
...	...

MEMORIA RAM

Il linguaggio Assembly

Altro esempio completo:

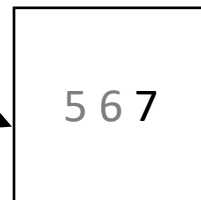
// metto il 5 in memoria all'indirizzo 6

@5
M=A

@6
M=D

//prelevo il 10

@7
D=M



A

M=A



D

M=D
D=M

Indirizzo	Valore
0	0
1	0
2	0
3	0
4	0
5	0
6	5
7	10
8	0
9	0
...	...

MEMORIA

Il linguaggio Assembly

Supponiamo di voler creare un programma che fa la somma di due variabili:

```
int a,b,c;  
b=8;  
c=5;  
a=b+c;
```

Linguaggio C

```
// a: indirizzo 0  
// b: indirizzo e 1 carico 8  
// c: indirizzo e 2 carico 5
```

```
// carico 8 in b  
@8  
D=A  
@1  
M=D
```

```
// carico 5 in c  
@5  
D=A  
@2  
M=D
```

```
// eseguo la somma  
@1  
D=M  
@2  
D=D+M  
@0  
M=D
```

```
// a: indirizzo 0
// b: indirizzo e 1 carico 8
// c: indirizzo e 2 carico 5
```

Possiamo fare in modo che sia l'assemblatore a decidere l'indirizzo di memoria delle variabili che andremo ad utilizzare.

```
// carico 8 in b
@8
D=A
@1
M=D
```

```
// carico 8 in b
@8
D=A
@b
M=D
```

```
// carico 5 in c
@5
D=A
@2
M=D
```

```
// carico 5 in c
@5
D=A
@c
M=D
```

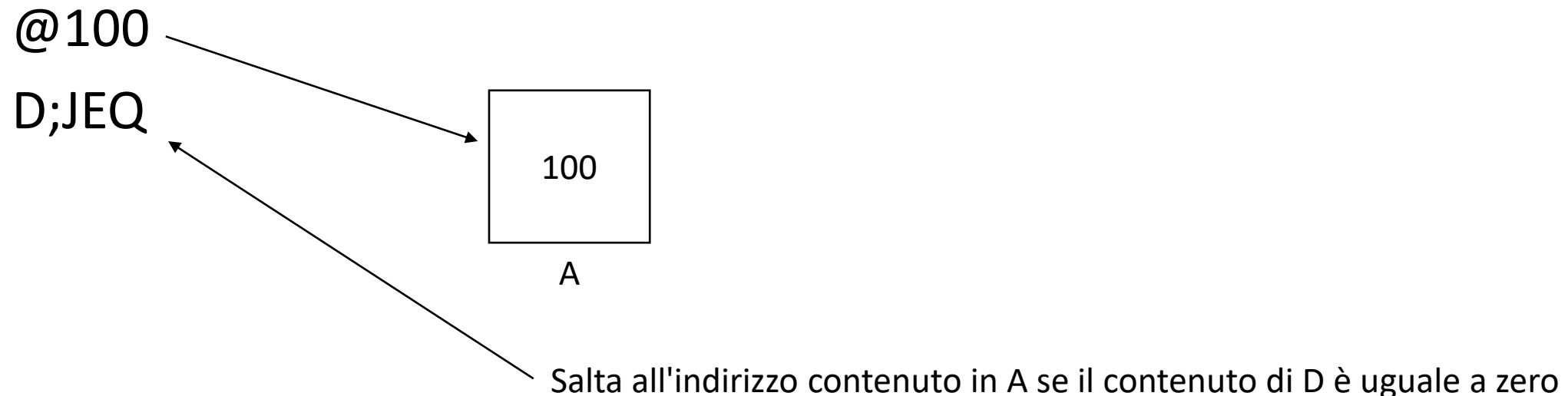
```
// eseguo la somma
@1
D=M
@2
D=D+M
@0
M=D
```

```
// eseguo la somma
@b
D=M
@c
D=D+M
@a
M=D
```

Il linguaggio Assembly

È possibile fare in modo che il nostro programma effettui dei salti tra le istruzioni al verificarsi di una condizione ben definita.

In linguaggio Assembly i salti servono per implementare sequenza e selezione dei classici linguaggi di programmazione (if – while- for ecc..).



Il linguaggio Assembly

Altre istruzioni di salto:

Mnemonic	Effect
null	No jump
JGT	If <i>out</i> > 0 jump
JEQ	If <i>out</i> = 0 jump
JGE	If <i>out</i> ≥ 0 jump
JLT	If <i>out</i> < 0 jump
JNE	If <i>out</i> ≠ 0 jump
JLE	If <i>out</i> ≤ 0 jump
JMP	Jump

Il linguaggio Assembly

Supponiamo di voler creare un programma che fa la somma dei numeri da 1 a 100 attraverso addizioni ripetute.

Tale programma scritto in C è il seguente:

```
int i = 1;  
int sum = 0;  
while (i<=100) {  
    sum+=i;  
    i++  
}
```

```
// decido che i ha indirizzo 0
// decido che sum ha indirizzo 1
@0      // Carica l'indirizzo di memoria i dentro A
M=1     // Carica il valore 1 nell'indirizzo di memoria specificato da A
@1      // Carica l'indirizzo di memoria 1 dentro A
M=0     // Carica il valore 0 nell'indirizzo di memoria specificato da A
(LLOOP)
@0
D=M     // Metto il valore di i nel registro D
@100    // Metto il valore 100 in A
D=D-A   // D=i-100
@END    // Carico l'indirizzo di memoria di END dentro A
D;JGT   // se (i-100)>0 vai a END
@0
D=M     // D=i
@1      // A=indirizzo di sum
M=D+M   // sum=sum+i
@0
M=M+1
@LOOP
0;JMP
(END)
@END
0;JMP   // Loop infinito
```

```
@i      // Carica l'indirizzo di memoria i dentro A
M=1     // Carica il valore 1 nell'indirizzo di memoria specificato da A
@sum    // Carica l'indirizzo di memoria 1 dentro A
M=0     // Carica il valore 0 nell'indirizzo di memoria specificato da A
(LOOP)
@i
D=M     // Metto il valore di i nel registro D
@100    // Metto il valore 100 in A
D=D-A   // D=i-100
@END    // Carico l'indirizzo di memoria di END dentro A
D;JGT   // se (i-100)>0 vai a END
@i
D=M     // D=i
@sum    // A=indirizzo di sum
M=D+M   // sum=sum+i
@i
M=M+1
@LOOP
0;JMP
(END)
@END
0;JMP   // Loop infinito
```

Versione con le variabili

Teoria sul linguaggio macchina

Esistono tre tipologie di istruzioni in linguaggio macchina:

1. Aritmetico-Logiche
2. Accesso alla memoria
3. Controllo di Flusso

Tipologie di istruzioni

Le Istruzioni **Aritmetico-Logiche** eseguono operazioni matematiche di base come somme e sottrazioni. In questa categoria rientrano anche le operazioni booleane, le negazioni, i bit shifting e così via.

Alcuni esempi:

```
ADD R2,R1,R3  // R2←R1+R3 where R1,R2,R3 are registers

ADD R2,R1,foo // R2←R1+foo where foo stands for the
               // value of the memory location pointed
               // at by the user-defined label foo.

AND R1,R1,R2   // R1←bit wise And of R1 and R2
```

Tipologie di istruzioni

Le Istruzioni di **Accesso alla Memoria** si suddividono in tre sottocategorie:

1. **Indirizzamento diretto:** specifico l'indirizzo di memoria direttamente nell'istruzione

```
LOAD R1,67    // R1←Memory[67]  
  
// Or, assuming that bar refers to memory address 67:  
LOAD R1,bar    // R1←Memory[67]
```

2. **Indirizzamento immediato**
3. **Indirizzamento indiretto**

Tipologie di istruzioni

Le Istruzioni di **Accesso alla Memoria** si suddividono in tre sottocategorie:

- 2. Indirizzamento immediato:** Questo metodo di indirizzamento consente di caricare il valore di una costante direttamente in un registro

```
LOADI R1,67    //  R1←67
```

Tipologie di istruzioni

Le Istruzioni di **Accesso alla Memoria** si suddividono in tre sottocategorie:

3. Indirizzamento indiretto: In questo caso l'indirizzo di memoria non è hard-coded nell'istruzione ma in quest'ultima viene specificata la locazione di memoria che contiene il valore. Supponiamo ad esempio di avere la seguente istruzione c: `x=foo[j]`; tale istruzione viene tradotta dal compilatore nel linguaggio macchina nel seguente modo:

```
// Translation of x=foo[j] or x=*(foo+j):  
ADD R1,foo,j    // R1←foo+j  
LOAD* R2,R1     // R2←Memory[R1]  
STR R2,x        // x←R2
```

Tipologie di istruzioni

Le Istruzioni di **Controllo di Flusso** sono tutte quelle che prevedono salti tra le istruzioni del programma.

Solitamente il salto tra le istruzioni del programma si verifica a fronte di una condizione.

Esempio:

High-level

```
// A while loop:  
while (R1>=0) {  
    code segment 1  
}  
code segment 2
```

Low-level

```
// Typical translation:  
beginWhile:  
    JNG R1,endWhile // If R1<0 goto endWhile  
    // Translation of code segment 1 comes here  
    JMP beginWhile  // Goto beginWhile  
endWhile:  
    // Translation of code segment 2 comes here
```

L'assemblatore (assembler)

L'assemblatore si occupa di tradurre le istruzioni mnemoniche (es: @100) in sequenze di zeri e uni.

Vediamo come funziona il nostro assemblatore.

L'assemblatore (assembler)

Le istruzioni si suddividono in due categorie principali:

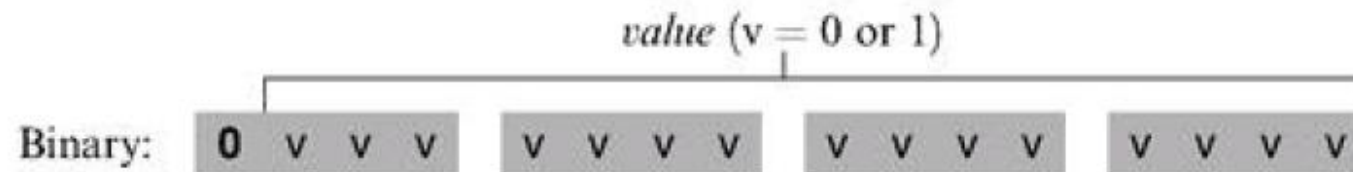
1. Istruzioni di tipo A: sono quelle che intervengono direttamente sul registro A e forzano un valore al suo interno (es: @100)
2. Istruzioni di tipo C: tutte le altre istruzioni

L'assemblatore (assembler)

Istruzioni di tipo A:

Dato che le istruzioni della nostra macchina sono lunghe tutte 16 bit, utilizziamo il primo bit per riconoscere il tipo di istruzione (A o C) e usiamo gli altri bit per indicare il valore da inserire in A.

A-instruction: @value // Where *value* is either a non-negative decimal number
// or a symbol referring to such number.



Esempio: @100 → 0000 0000 0110 0100

L'assemblatore (assembler)

Istruzioni di tipo C:

Sono il cavallo di battaglia della nostra macchina e fanno praticamente tutto loro.

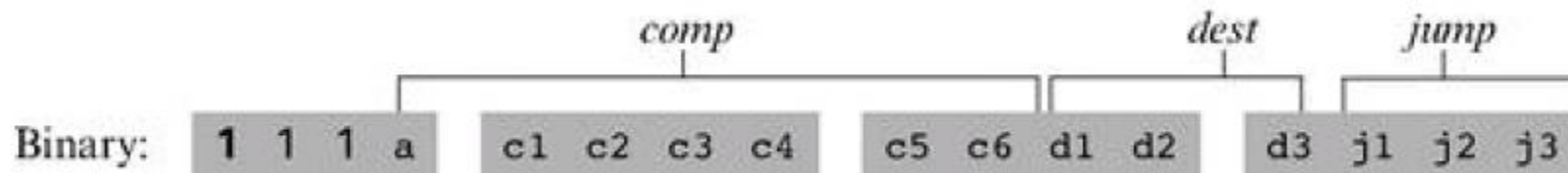
Tali istruzioni contengono al loro interno le seguenti informazioni:

- a) Cosa calcolare
- b) Dove salvare il risultato
- c) Cosa fare successivamente

L'assemblatore (assembler)

Istruzioni di tipo C:

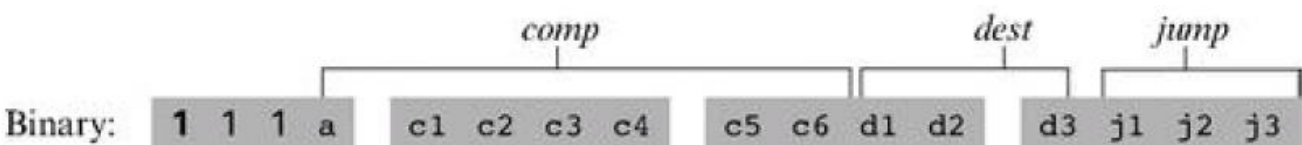
Questo è lo schema di un'istruzione di tipo C:



Il bit più a sinistra deve essere 1 per distinguerle dalle istruzioni di tipo A, mentre i successivi due bit non sono utilizzati (noi li mettiamo a 1).

L'assemblatore (assembler)

Istruzioni di tipo C: campo comp



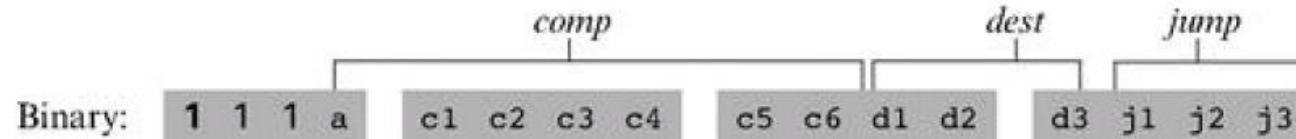
Il bit a e i 6 bit del campo comp vengono collegati ai piedini di controllo della ALU che abbiamo costruito e permettono di fare le seguenti operazioni elencate nella tabella accanto. Supponiamo di voler calcolare D-1, allora devo settare i bit nel seguente modo: a=0, c1=0, C2=0, c3=1, c4=1, c5=1, c6=0.

L'istruzione al momento è così tradotta:
1110001110dddj

(when a=0) comp mnemonic	c1	c2	c3	c4	c5	c6	(when a=1) comp mnemonic
0	1	0	1	0	1	0	
1	1	1	1	1	1	1	
-1	1	1	1	0	1	0	
D	0	0	1	1	0	0	
A	1	1	0	0	0	0	M
!D	0	0	1	1	0	1	
!A	1	1	0	0	0	1	!M
-D	0	0	1	1	1	1	
-A	1	1	0	0	1	1	-M
D+1	0	1	1	1	1	1	
A+1	1	1	0	1	1	1	M+1
D-1	0	0	1	1	1	0	
A-1	1	1	0	0	1	0	M-1
D+A	0	0	0	0	1	0	D+M
D-A	0	1	0	0	1	1	D-M
A-D	0	0	0	1	1	1	M-D
D&A	0	0	0	0	0	0	D&M
D A	0	1	0	1	0	1	D M

L'assemblatore (assembler)

Istruzioni di tipo C: campo dest



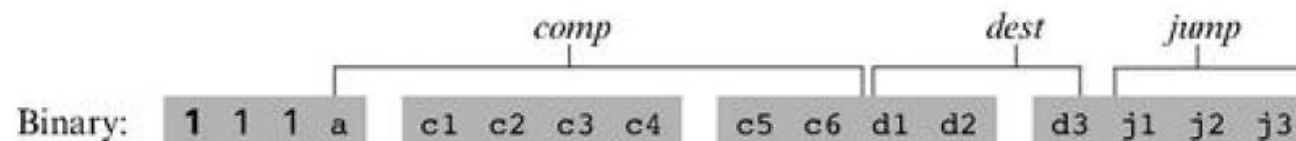
Il bit del campo dest specificano dove voglio salvare il risultato in accordo con la tabella sottostante:

d1	d2	d3	Mnemonic	Destination (where to store the computed value)
0	0	0	null	The value is not stored anywhere
0	0	1	M	Memory[A] (memory register addressed by A)
0	1	0	D	D register
0	1	1	MD	Memory[A] and D register
1	0	0	A	A register
1	0	1	AM	A register and Memory[A]
1	1	0	AD	A register and D register
1	1	1	AMD	A register, Memory[A], and D register

Supponiamo di voler salvare il risultato di D-1 in D allora la nostra istruzione diventerà la seguente: 11100011100**10**j1j2j3

L'assemblatore (assembler)

Istruzioni di tipo C: campo jump



Il bit del campo jump specificano come voglio eseguire il salto tra le istruzioni in accordo con la seguente tabella:

j1 (<i>out</i> < 0)	j2 (<i>out</i> = 0)	j3 (<i>out</i> > 0)	Mnemonic	Effect
0	0	0	null	No jump
0	0	1	JGT	If <i>out</i> > 0 jump
0	1	0	JEQ	If <i>out</i> = 0 jump
0	1	1	JGE	If <i>out</i> ≥ 0 jump
1	0	0	JLT	If <i>out</i> < 0 jump
1	0	1	JNE	If <i>out</i> ≠ 0 jump
1	1	0	JLE	If <i>out</i> ≤ 0 jump
1	1	1	JMP	Jump

Supponiamo di non voler eseguire salti, allora la nostra istruzione diventerà la seguente: 1110001110010000

Esercizi in linguaggio macchina

1. Fare un programma che salva due numeri in memoria ed esegue la sottrazione tra di essi.
2. Fare un programma che salva due numeri in memoria ed esegue la moltiplicazione tra di essi.
3. Fare un programma che salva due numeri in memoria ed esegue la divisione tra di essi, salvando il quoziente in una apposita variabile e il resto da in un'altra.

Esercizi in linguaggio macchina

OPZIONALE:

Tradurre gli esercizi fatti in precedenza in sequenze di zeri e uni manualmente.